



Movie Search

Sofia G. und Sofia W.
30.01.2026

Objektorientierte Programmierung 1 WiSe 2526





Gliederung



01 Aufgabenstellung

02 Ordnerstruktur

03 CSV Datei

0

4

Cara Aufgabe

05 Graf Aufgabe

06 Weber Aufgabe

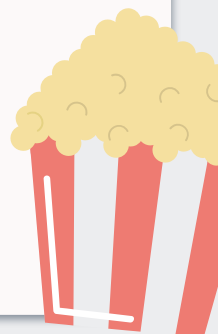
07 Demo





(01) Aufgabenstellung

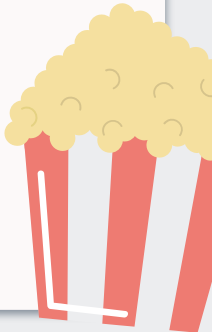
- Erstellung eines Konsolenprogramms zur Suche in der IMDB-Datenbank
- Kriterien:
 - Suche nach Titel oder Regisseur einschränken.
 - nach einem Teil des Bezeichners suchen.
 - IMDB CSV Datei einlesen
 - Ein Menü erstellen mit folgenden Optionen zur Auswahl:
 - a) Neue Suche
 - b) Neue Filme eingeben
 - c) Programm beenden.
 - Die eingegebenen Filme werden in einer separaten Textdatei gespeichert, die für die Suche zusätzlich geladen wird.





(2) Ordnerstruktur

```
├── data
├── MovieSearch
│   ├── resources
│   │   ├── data
│   │   │   ├── movie_directors_export_new.csv
│   │   │   └── user_movies.csv
│   │   └── media
│   │       └── gifs
│   │           └── Movie_Search_Demonstration.GIF
│   └── src
│       ├── model
│       │   └── Movie.java
│       ├── repository
│       │   └── MovieRepository.java
│       ├── search
│       │   └── MovieSearchService.java
│       ├── storage
│       │   └── CsvUserMovieStorage.java
│       └── ui
│           └── MovieSearchApp.java
├── OOP1_Projekt_Movie_Search.jar
└── README.md
```

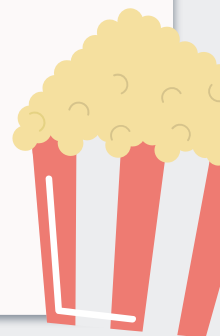




(03) CSV-Datei

	A	B	C	D	E	F	G	H
1	tt0000009	Miss Jerry	Alexander Black					
2	tt0000147	The Corbett-Fitzsimmons Fight	Enoch J. Rector					
3	tt0000335	Soldiers of the Cross	Herbert Booth					
4	tt0000335	Soldiers of the Cross	Joseph Perry					
5	tt0000502	Bohemios	Ricardo de Ba??os					
6	tt0000574	The Story of the Kelly Gang	Charles Tait					
7	tt0000615	Robbery Under Arms	Charles MacMahon					
8	tt0000630	Amleto	Mario Caserini					
9	tt0000675	Don Quijote	Narciso Cuy??s					
10	tt0000676	Don ??lvaro o la fuerza del sino	Narciso Cuy??s					
11	tt0000679	The Fairylogue and Radio-Plays	Francis Boggs					
12	tt0000679	The Fairylogue and Radio-Plays	Otis Turner					
13	tt0000739	El pastorcito de Torrente	Antonio Cuesta					
14	tt0000793	Andreas Hofer	Rudolf Biebrach					
15	tt0000812	El bloqueo Velarde	Ignacio Coyne					
16	tt0000814	La bocana de Mar Chica	Ignacio Coyne					
17	tt0000838	A Cultura do Cacau	Ernesto de Albuquerque					
18	tt0000842	De Garraf a Barcelona	Fructu??s Gelabert					
19	tt0000846	Un d??a en Xochimilco	Hermanos Alva					
20	tt0000850	Los dos hermanos	Ricardo de Ba??os					
21	tt0000850	Los dos hermanos	Alberto Marro					
22	tt0000859	Fabricaci??n del corcho en Sant Feliu de Guixols	Fructu??s Gelabert					
23	tt0000862	Faldgruben	Emanuel Tvede					
24	tt0000867	Fiesta de toros	Hermanos Alva					
25	tt0000868	Fiestas de Santa Luc??a	Belenes	Fructu??s Gelabert				
26	tt0000869	Fiestas en La Garriga	Fructu??s Gelabert					
27	tt0000879	Gira pol??tica de Madero y Pino Su??rez	Hermanos Alva					
28	tt0000886	Hamlet	G??rard Bourgeois					

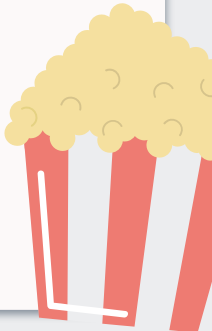
< > movie_directors_export_new +





(04) Kietzig Aufgabe: MovieSearchApp (UI)

1. Zweck: Konsolenprogramm mit Menu
2. Methoden:
 - 2.1. `public void run()`
 - 2.2. `private void printMenu()`
 - 2.3. `private void handleSearch()`
 - 2.4. `private void handleAddMovie()`
3. Wird gebraucht von:
 - 3.1. `MovieRepository` (laden, liste holen, hinzufügen)
 - 3.2. `MovieSearchService` (suchen)
 - 3.3. `CsvUserMovieStorage` (speichern)
 - 3.4. `Movie` (Objekt bauen)

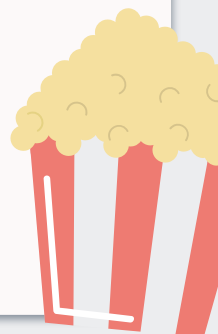




(04) Kietzig Aufgabe: MovieSearchApp (UI)

4. Braucht (ruft):

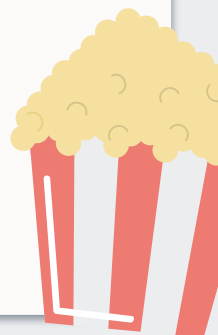
- `MovieRepository.loadImdbCsv(path)`
- `MovieRepository.loadUserCsv(path)`
- `MovieRepository.getAllMovies()`
- `MovieRepository.addUserMovie(movie)`
- `MovieSearchService.search(movies, titlepart, directorPart)`
- `CsvUserMovieStorage.append(movie)`
- `new Movie(imdbId,title,director)`





(04) Kietzig Aufgabe: `CsvUserMovieStorage`

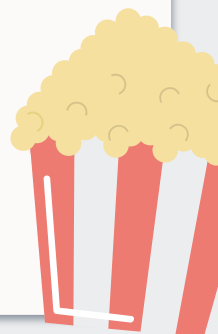
1. Zweck: Schreibt neue Filme dauerhaft in `user_movies.csv`
2. Methoden:
 - 2.1. `CsvUserMovieStorage(String userCsvPath)`
 - 2.2. `CsvUserMovieStorage(String userCsvPath)`
3. Wird gebraucht von:
 - 3.1. `MovieSearchApp` beim Hinzufügen von Filmen
4. Braucht (ruft):
 - 4.1. `Movie.getImdbld()`, `Movie.getTitle()`, `Movie.getDirector()`





(05) Graf Aufgabe: MovieSearchService

1. Zweck: Regelt die Suchlogik
2. Methoden:
 - 2.1. `public List<Movie> search(List<Movie> movies, String titlePart, String directorPart)`
3. Wird gebraucht von:
 - 3.1. MovieSearchApp bei der Suche
4. Braucht (ruft): `Movie.getTitle()` und `Movie.getDirector()` von der Movie Klasse





```
7 public class MovieSearchService {
8
9     public List<Movie> search(List<Movie> movies, String titlePart, String directorPart) {
10
11         List<Movie> result = new ArrayList<>();
12
13         if (movies == null) {
14             return result;
15             // Wenn unser ArrayList fuer Movies leer ist
16         }
17
18
19         String titleFilter;
20         if (titlePart == null) {
21             titleFilter = "";
22         } else {
23             titleFilter = titlePart.trim().toLowerCase();
24         }
25
26         String directorFilter;
27         if (directorPart == null) {
28             directorFilter = "";
29         } else {
30             directorFilter = directorPart.trim().toLowerCase();
31         }
32
33         // Die Suche muss case-insensitive sein also .toLowerCase() verwenden.
34         // Die .trim() methode entfernt white space bei der Eingabe des Benutzers "methode
35         // Dies behandelt leere Eingaben vom Benutzer und filter Eingaben
```



```
37     for (Movie movie : movies) {
38         // Dies geht durch alle Filme durch
39
40         boolean matchesTitle;
41         boolean matchesDirector;
42
43         // Nach Titel suchen und pruefen
44         if (titleFilter.isEmpty()) {
45             matchesTitle = true;
46         } else {
47             String movieTitle = movie.getTitle();
48
49             if (movieTitle == null) {
50                 matchesTitle = false;
51
52             } else {
53                 matchesTitle = movieTitle.toLowerCase().contains(titleFilter);
54             }
55         }
56     }
```

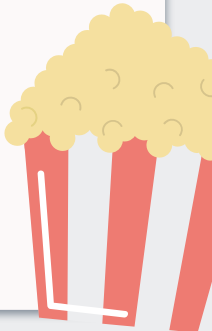


```
57 // nach Director suchen und pruefen
58 if (directorFilter.isEmpty()) {
59     matchesDirector = true;
60 } else {
61     String movieDirector = movie.getDirector();
62
63     if (movieDirector == null) {
64         matchesDirector = false;
65     } else {
66         matchesDirector = movieDirector.toLowerCase().contains(directorFilter);
67     }
68 }
69
70 if (matchesTitle && matchesDirector) {
71     result.add(movie);
72     // Kombinationssuche erfolgt nur wenn Titel und Director passen bzw. true sind
73     // erlaubt auch Suche nur nach Director oder Titel wenn eines davon leer ist.
74 }
75 }
76
77 return result;
78 }
79 }
```



(05) Graf Aufgabe: GitHub Repo

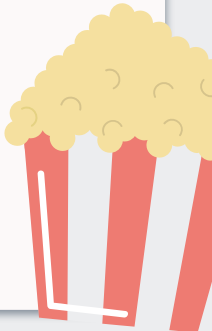
1. Erstellung und Instandhaltung des Repositories
2. README.md-Datei schreiben
 - 2.1. General
 - 2.2. Root Folder
 - 2.3. Rollen
 - 2.4. Bedienungs- und Installationsanweisung
 - 2.5. Video und GIFs erstellen





(06) Weber Aufgaben

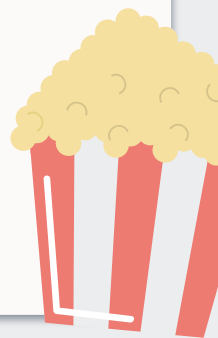
1. **Movie.java**
2. MovieRepository.java - Imports und Start
3. MovieRepository.java - Methode loadImdbCsv
4. MovieRepository.java - Methode loadUserCsv
5. MovieRepository.java - Methode addUserMovie und getAllMovies





Movie.java X

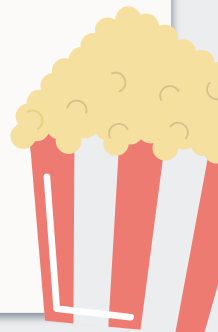
```
1 package model;
2
3 public class Movie {
4     private final String imdbId;
5     private final String title;
6     private final String director;
7
8     public Movie(String imdbId, String title, String director) {
9         this.imdbId = imdbId;
10        this.title = title;
11        this.director = director;
12    }
13
14    public String getImdbId() { return imdbId; }
15    public String getTitle() { return title; }
16    public String getDirector() { return director; }
17
18    @Override
19    public String toString() {
20        return title + " - " + director;
21    }
22 }
```





(06) Weber Aufgabe: Movie.java

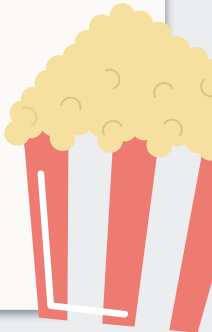
1. Repräsentiert einen einzelnen Film als Datenobjekt
2. Wird gebraucht von:
 - 2.1. MovieRepository
 - 2.2. MovieSearchService
 - 2.3. MovieSearchApp
 - 2.4. CsvUserMovieStorage
3. Braucht (ruft): reine Modellklasse, keine Zugriffe auf andere Klassen des Projekts





(06) Weber Aufgaben

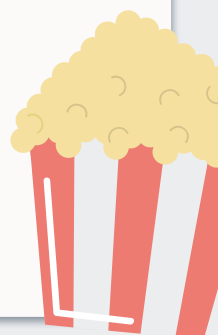
1. Movie.java
2. **MovieRepository.java - Imports und Start**
3. MovieRepository.java - Methode loadImdbCsv
4. MovieRepository.java - Methode loadUserCsv
5. MovieRepository.java - Methode addUserMovie und getAllMovies





MovieRepository.java X

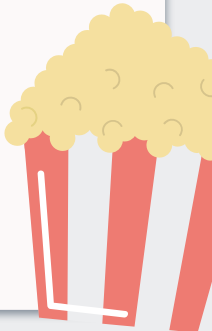
```
1 package repository;
2
3 import model.Movie;
4
5 import java.util.ArrayList;
6 import java.util.Collections;
7 import java.util.List;
8 import java.io.BufferedReader;
9 import java.io.File;
10 import java.io.FileInputStream;
11 import java.io.InputStreamReader;
12 import java.nio.charset.StandardCharsets;
13
14 public class MovieRepository {
15
16     private final List<Movie> movies = new ArrayList<>();
17 }
```



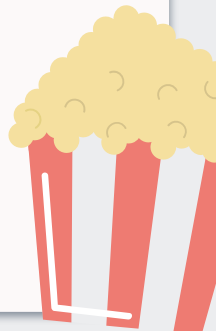


(06) Weber Aufgaben

1. Movie.java
2. MovieRepository.java - Imports und Start
3. **MovieRepository.java - Methode loadImdbCsv**
4. MovieRepository.java - Methode loadUserCsv
5. MovieRepository.java - Methode addUserMovie und getAllMovies



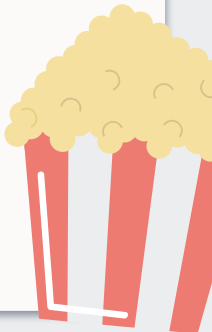
```
17
18 public void loadImdbCsv(String path) throws Exception {
19
20     File file = new File(path);
21
22     // File Input liest Datei in Bytes,
23     // InputStreamReader wandelt in UTF-8 um (bspw. bessere Darstellung von Umlauten),
24     // BufferedReader erlaubt effizientes Lesen (readline-Methode)
25     BufferedReader reader = new BufferedReader(
26         new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF_8));
27
28     String line;
29
30     while ((line = reader.readLine()) != null) {
31
32         // Leere Zeilen überspringen
33         if (line.trim().isEmpty()) {
34             continue;
35         }
36
37         // CSV-Zeile aufteilen
38         String[] parts = line.split(",");
39
40         // Ungültige Zeilen überspringen
41         if (parts.length < 3) {
42             continue;
43         }
44
45         String imdbId = parts[0].trim();
46         String title = parts[1].trim();
47         String director = parts[2].trim();
48
49         // Pflichtfelder prüfen
50         if (imdbId.isEmpty() || title.isEmpty() || director.isEmpty()) {
51             continue;
52         }
53         // Erstellung eines Movie Objekts und hinzufügen in movies-Array
54         Movie movie = new Movie(imdbId, title, director);
55         movies.add(movie);
56     }
57
58     reader.close();
59 }
60
```



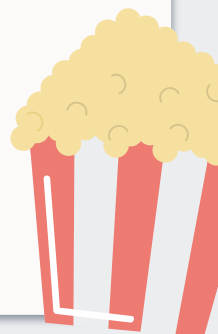


(06) Weber Aufgaben

1. Movie.java
2. MovieRepository.java - Imports und Start
3. MovieRepository.java - Methode loadImdbCsv
4. **MovieRepository.java - Methode loadUserCsv**
5. MovieRepository.java - Methode addUserMovie und getAllMovies



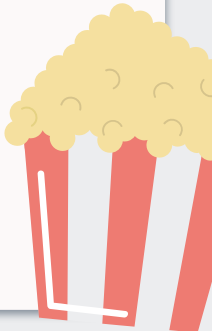
```
60
61 public void loadUserCsv(String path) throws Exception {
62
63     File file = new File(path);
64
65     // Wenn Datei nicht existiert: nichts tun
66     if (!file.exists()) {
67         return;
68     }
69
70     BufferedReader reader = new BufferedReader(
71         new InputStreamReader(new FileInputStream(file), StandardCharsets.UTF_8));
72
73     String line;
74
75     while ((line = reader.readLine()) != null) {
76
77         if (line.trim().isEmpty()) {
78             continue;
79         }
80
81         String[] parts = line.split(",");
82
83         if (parts.length < 3) {
84             continue;
85         }
86
87         String imdbId = parts[0].trim();
88         String title = parts[1].trim();
89         String director = parts[2].trim();
90
91         if (imdbId.isEmpty() || title.isEmpty() || director.isEmpty()) {
92             continue;
93         }
94
95         Movie movie = new Movie(imdbId, title, director);
96         movies.add(movie);
97     }
98
99     reader.close();
100 }
101
```





(06) Weber Aufgaben

1. Movie.java
2. MovieRepository.java - Imports und Start
3. MovieRepository.java - Methode loadImdbCsv
4. MovieRepository.java - Methode loadUserCsv
5. **MovieRepository.java - Methode addUserMovie und getAllMovies**





MovieRepository.java ×

101

102

103

104

105

106

107

108

109

110

public void addUserMovie(Movie movie) {

 movies.add(movie);

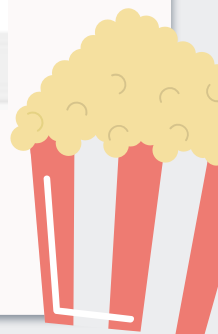
}

public List<Movie> getAllMovies() {

return Collections.unmodifiableList(movies);

}

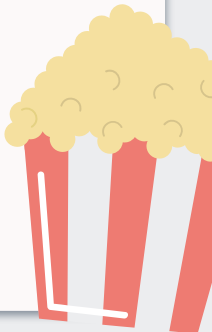
}





(06) Weber Aufgabe: MovieRepository.java

1. Verwaltet alle Filme im Speicher (IMDB-Filme und Benutzer-Filme).
2. Wird gebraucht von:
 - 2.1. MovieSearchApp
3. Braucht (ruft):
 - 3.1. Movie (Konstruktor, Getter)
 - 3.2. java.util.List, java.util.ArrayList, java.util.Collections
 - 3.3. java.io.* (Dateizugriff, CSV-Lesen)

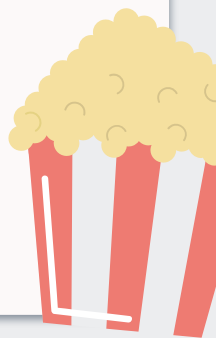
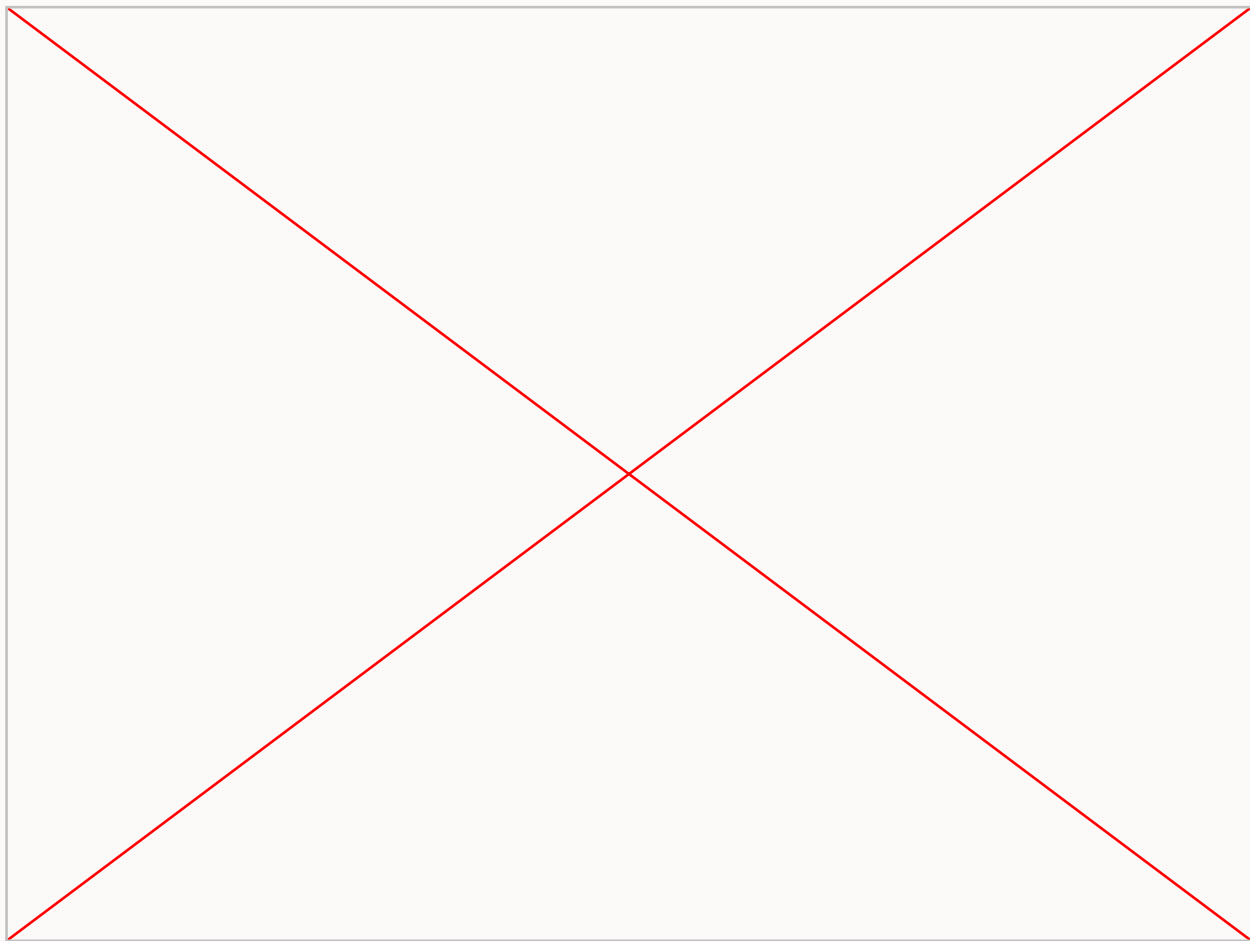




07

Demo

VScode





Quellenverzeichnis

- slidesgo.com (2025) *Movie Clapperboard Theme, Movie Clapperboard Theme Presentation*. Verfügbar unter:
<https://slidesgo.com/theme/movie-clapperboard-theme> (Zugegriffen: 29. Januar 2026).

